Meta 3 — Transfer Learning e Aplicação Web

Reconhecimento de Emoções Faciais

Autores: Ana Rita Conceição Pessoa e David José Gomes de Matos

Instituto Superior de Engenharia do Instituto Politécnico de Coimbra

Departamento de Informática e Sistemas

😠 😨 😊 😢 😲

FASE III – APRENDIZAGEM POR TRANSFERÊNCIA E IMPLEMENTAÇÃO DE UMA APLICAÇÃO

Coimbra, dezembro de 2025

Enquadramento do Projeto

O projeto desenvolve um sistema de reconhecimento de emoções faciais ao longo de três metas, com cinco classes principais: anger, fear, happy, sad e surprise.

Meta 1

CNN construída de raiz (from scratch)

→ Base: datasets sintéticos

Meta 2

Otimização via PSO/GSA

→ Refinamento da top layer

Meta 3

Transfer Learning + Otimização + Web App

→ MobileNetV2 + Gradio

Classes de Emoção:

- Anger
- Fear
- Happy
- Sad
- Surprise

Objetivos Específicos da Meta 3

Esta meta visa desenvolver um classificador de emoções faciais robusto através de transferência de aprendizagem e otimização de hiperparâmetros.

Usar MobileNetV2 como base pré-treinada

Implementar transferência de aprendizagem com uma arquitetura pré-treinada em larga escala, carregando pesos prévios e mantendo a base convolucional congelada.

Ajustar top layers

Construir uma pequena rede densa sobre a base convolucional, composta por Global Average Pooling, camadas densas com ativação ReLU e uma camada final softmax.

Aplicar otimizações de hiperparâmetros

Testar Grid Search, Random Search e Particle Swarm Optimization (PSO) para otimizar a top layer, ajustando número de neurónios, taxa de dropout e taxa de aprendizagem.

Avaliar impacto do tamanho do dataset

Realizar estudos experimentais com subconjuntos progressivos do dataset (25%, 50%, 75% e 100% das instâncias de treino) para analisar a curva de aprendizagem.

Construir aplicação web com Gradio

Desenvolver uma interface web que permita carregar imagens ou utilizar a webcam, processando-as com o modelo otimizado e apresentando a emoção prevista com as respetivas probabilidades.

Arquitetura Geral do Sistema



Pipeline de Dados e Fluxo do Projeto

Fluxo completo desde carregamento de dados até integração na aplicação web.

Carregamento de Dados

- ImageDataGenerator
- Divisão train/validation/test

Pré-processamento

- Redimensionamento (224×224)
- Normalização

Treino (TL)

- MobileNetV2 pré-treinada
- Top layer customizada

Otimização

- Grid Search
- Random Search
- PSO

Ajuste de Hiperparâmetros

- Hidden units
- Dropout
- Learning rate

Avaliação

- Teste no conjunto real
- Métricas de performance

Integração Web

- Gradio
- Interface user-friendly

Dataset: Distribuição por Classe

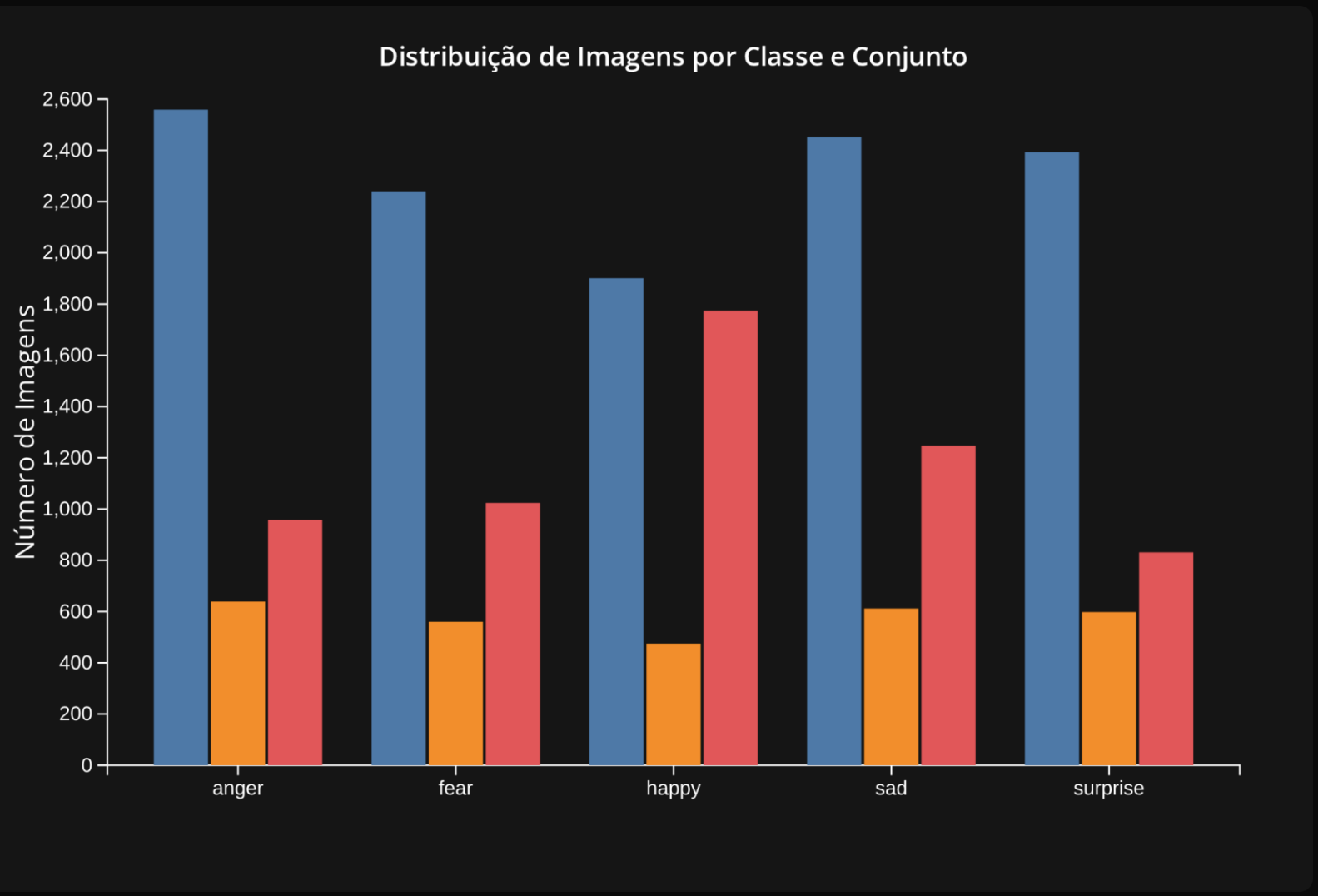
Análise da distribuição das cinco classes de emoções nos conjuntos de treino, validação e teste.

Distribuição Observada

- Há diferenças moderadas no número de imagens por classe
- A classe **sad** é ligeiramente a mais representada
- As classes **anger** e **fear** têm menos amostras
- ⚠ Este desbalanceamento pode influenciar o desempenho do modelo

Legenda:

Train Validation Test



Preparação dos Dados

Redimensionamento
224×224 píxeis

Normalização
via preprocess_input

Class Weighting
Balanceamento

Técnicas de Pré-processamento

- Redimensionamento para 224×224 píxeis, tamanho requerido pela MobileNetV2
- Normalização através da função preprocess_input da MobileNetV2
- Sem data augmentation avançada, para isolar o impacto da TL e otimização

Informações Adicionais

- Dataset não totalmente equilibrado, com diferenças significativas no número de instâncias por classe
- Classe "sad" ligeiramente mais representada (4311 imagens)
- Classe "fear" e "surprise" têm menos instâncias (3824 e 3822)

Foram aplicados pesos inversamente proporcionais à frequência de cada classe durante o treino para mitigar o desbalanceamento.

Arquitetura do Modelo (Transfer Learning)

Base Pré-treinada (MobileNetV2)

- Camadas congeladas (frozen)
- Entrada: 224×224×3
- Saída: tensor 7×7×1280
- Arquitetura: MobileNetV2 (ImageNet)

Configuração do Modelo

- Número de neurônios: variável (hiperparâmetro)
- Taxa de dropout: variável (hiperparâmetro)
- Ativação: ReLU (intermédia), softmax (final)

Top Layer

GlobalAveragePooling2D
Reduz tensor 7×7×1280 para 1×1×1280

Dense (hidden units)
Camada oculta com número variável de neurônios

Dropout
Taxa ajustável para prevenir overfitting

Dense(5, softmax)
5 neurónios com ativação softmax para classificação

```python
base_model = MobileNetV2(
    include_top=False,
    weights="imagenet",
    input_shape=(IMG_SIZE, IMG_SIZE, 3)
)

for layer in base_model.layers:
    layer.trainable = False
```

```python
def build_default_model(hidden_units=256, dropout_rate=0.5, lr=1e-4):
    """
    Modelo default da Meta 3.
    Hidden_units e dropout_rate depois vão ser alvo de otimização na Grid, Random e PSO.
    """
    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(hidden_units, activation="relu")(x)
    x = Dropout(dropout_rate)(x)
    outputs = Dense(NUM_CLASSES, activation="softmax")(x)

    model = Model(inputs=base_model.input, outputs=outputs)
```

Impacto do Número de Instâncias

Análise do desempenho com diferentes percentagens do dataset e limitações do domain shift

```python
fractions = [0.25, 0.50, 0.75, 1.0]
acc_results = []

# Guardar o número total de passos originais
original_steps = train_gen.samples // BATCH_SIZE

for frac in fractions:
    print(f"\nTreinando com {frac*100}% dos dados...")

    # 1. Determinar quantos passos correspondem a esta fração
    steps_fraction = int(original_steps * frac)
    if steps_fraction < 1: steps_fraction = 1 # Garantir pelo menos 1 passo

    # 2. Construir um modelo NOVO (reset aos pesos)
    temp_model = build_default_model()

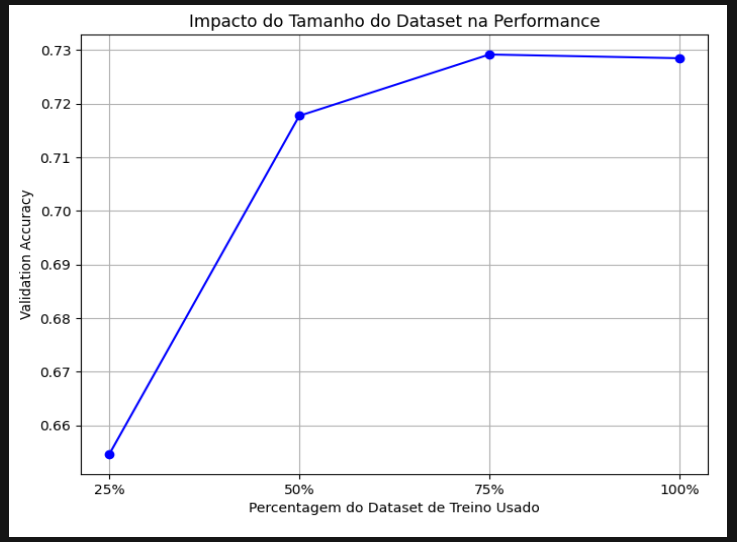
    # 3. Treinar com número limitado de passos
    temp_hist = temp_model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=5,
        steps_per_epoch=steps_fraction,
        class_weight=class_weights_dict,
        verbose=0
    )

    # 4. Avaliar no conjunto de validação (ou teste)
    val_loss, val_acc = temp_model.evaluate(val_gen, verbose=0)
    acc_results.append(val_acc)
```

Passagem de **25% para 50%** dos dados: aumento de **~6%** de accuracy (de ~65% para ~71%)

Estabilização

Entre **75% e 100%** dos dados, a curva estabiliza em torno de **72-73%,** indicando ponto de saturação



Limitação Principal

Arquitetura MobileNetV2 é eficiente na extração de características, mas o fator limitante é a qualidade e representatividade do dataset

Domain Shift

Principal desafio: diferença entre dados sintéticos (treino) e reais (teste), limitando a generalização do modelo

Nota

Resultados exploratórios do impacto do tamanho do treino.
O desempenho do modelo final é apresentado no slide seguinte.

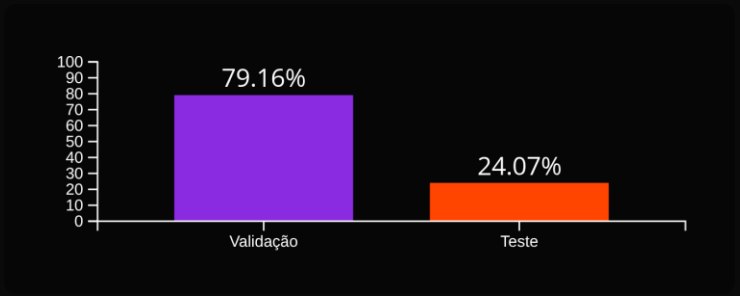
Resultados do Modelo Default

Modelo base com transferência de aprendizagem (MobileNetV2)

Validação
79,16%
Accuracy

Teste
24,07%
Accuracy

Domain Shift



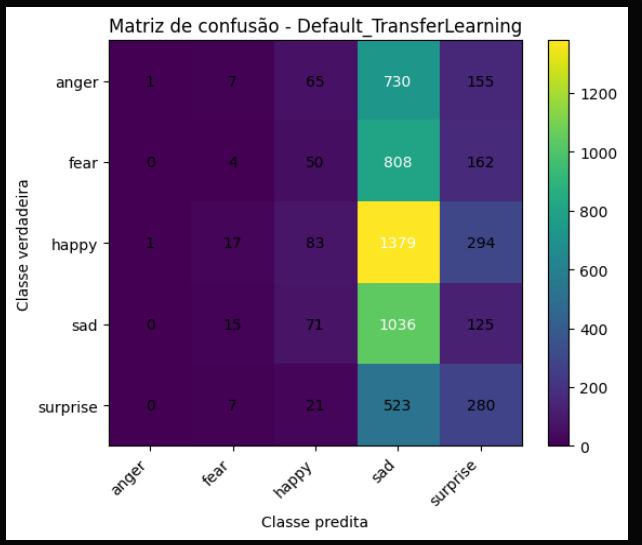

```python
from sklearn.utils.class_weight import compute_class_weight

# 1. Calcular pesos baseados na frequência das classes no treino
cls_train = train_gen.classes
class_weights = compute_class_weight(
    class_weight='balanced',
    classes=np.unique(cls_train),
    y=cls_train
)

# 2. Converter para formato exigido pelo Keras
class_weights_dict = dict(enumerate(class_weights))
print("Pesos das classes calculados (Balanceamento):", class_weights_dict)

# 3. Treinar com o argumento class_weight
history = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=EPOCHS,
    class_weight=class_weights_dict
)
```

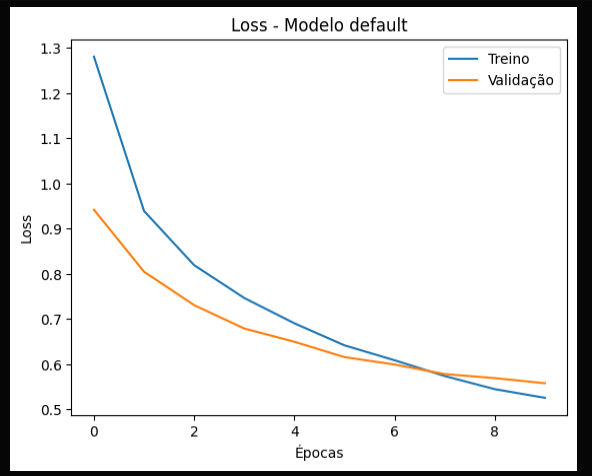
Matriz de Confusão

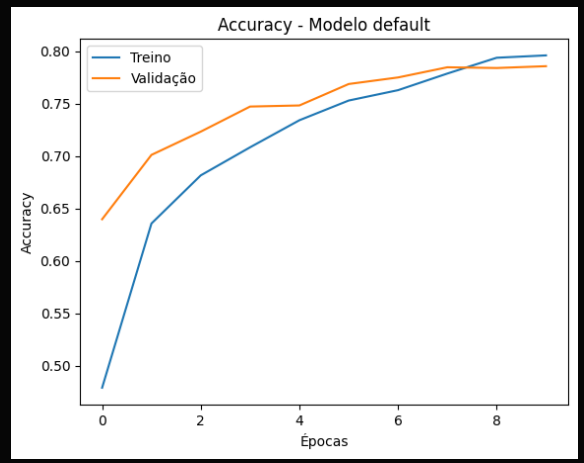


Análise

- Modelo treinado com técnica de Class Weighting para mitigar desbalanceamento
- Quebra acentuada de desempenho entre validação (79%) e teste (24%)
- Tendência a prever classes sad e surprise devido a desbalanceamento
- Loss de teste (**2,87**) inferior aos modelos otimizados, indicando menor overfitting

Curvas de Aprendizagem





Grid Search

Otimização de hiperparâmetros com 12 combinações avaliadas

Melhor Combinação

- Hidden Units: **512**
- Dropout: **0.3**
- Learning Rate: **0.0001**

Resultados

- Val Accuracy: **0.7916**
- Test Accuracy: **~0.235**

```python
for hidden_units in hidden_units_list:
    for dropout_rate in dropout_list:
        for lr in lr_list:
            comb_index += 1
            print("\n")
            print("=" * 60)
            print(f"Combinação {comb_index}/{total_combs}")
            print(f"hidden_units={hidden_units}, dropout={dropout_rate}, lr={lr}")
            print("=" * 60)

            K.clear_session()
            tf.random.set_seed(SEED)
            np.random.seed(SEED)

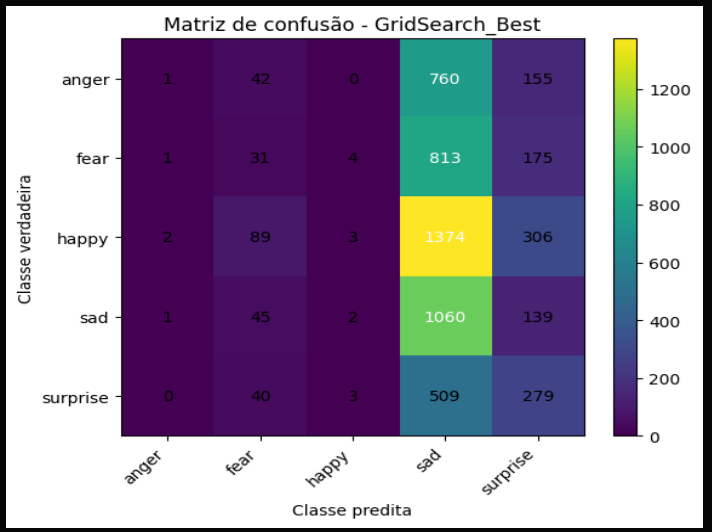
            model = build_model(
                hidden_units=hidden_units,
                dropout_rate=dropout_rate,
                lr=lr
            )

            history = model.fit(
                train_gen,
                validation_data=val_gen,
                epochs=EPOCHS,
                verbose=1
            )
```

Top 6 Resultados da Grid Search

Hidden Units	Dropout	LR	Val Accuracy
512	0.3	0.0001	**0.7916**
128	0.3	0.001	0.7909
256	0.3	0.001	0.7909
256	0.5	0.001	0.7871
128	0.5	0.001	0.7812
512	0.5	0.0001	0.7802

Matriz de Confusão



Random Search

10 iterações aleatórias de hiperparâmetros

Implementação

A Random Search sorteou valores aleatórios para os hiperparâmetros em cada iteração, explorando zonas do espaço de procura que a Grid Search não atinge.

Melhor Configuração

Hidden Units	Dropout	Learning Rate
192	0.36	0.00076

Resultados

Val Accuracy	Test Accuracy
0.7968	0.235

Top 5 Resultados

Iteração	Hidden Units	Dropout	Learning Rate	Val Accuracy
3	192	0.357	0.00076	0.7968
7	448	0.338	0.00095	0.7913
10	256	0.528	0.00041	0.7913
8	448	0.452	0.00017	0.7888
9	448	0.524	0.00021	0.7826

```python
N_TRIALS = 10  # número de experiências aleatórias
hidden_units_range = (128, 512)       # intervalo de inteiros
dropout_range      = (0.3, 0.6)       # intervalo contínuo
lr_log_range       = (1e-4, 1e-2)     # intervalo em escala log10

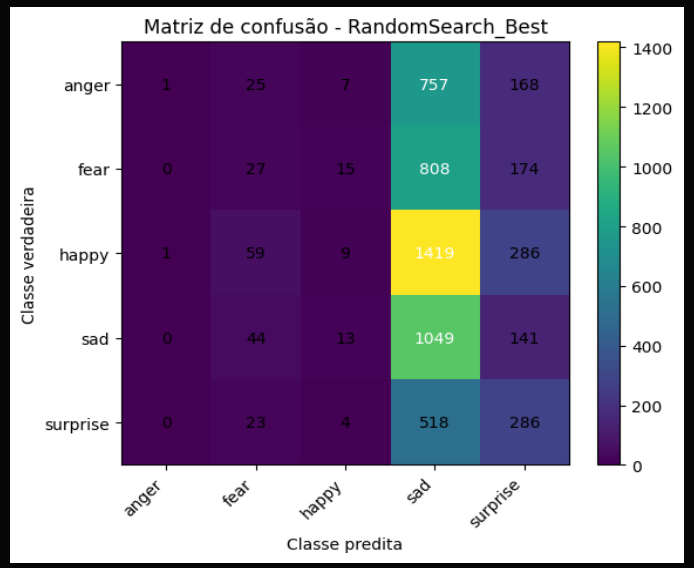
def sample_hyperparameters():
    """
    Amostra aleatoriamente uma combinação de hiperparâmetros.
    """
    # nº de unidades
    hu = random.randint(2, 8) * 64   # valores possíveis: 128,192,...,512
    hu = max(hidden_units_range[0], min(hu, hidden_units_range[1]))

    # dropout contínuo entre 0.3 e 0.6
    dr = random.uniform(dropout_range[0], dropout_range[1])

    # learning rate
    log_lr_min = np.log10(lr_log_range[0])
    log_lr_max = np.log10(lr_log_range[1])
    log_lr = random.uniform(log_lr_min, log_lr_max)
    lr = 10 ** log_lr

    return hu, dr, lr
```

Matriz de Confusão



Particle Swarm Optimization (PSO)

Otimização de hiperparâmetros, onde partículas exploram o espaço de soluções com base na melhor experiência individual e global.

Implementação

O PSO é um método de otimização inspirado na Natureza, no qual cada partícula representa uma combinação de hiperparâmetros e é atualizada iterativamente com base na melhor solução individual e global.

Melhor Configuração

Hidden Units	Dropout	Learning Rate
448	0.33	0.0007

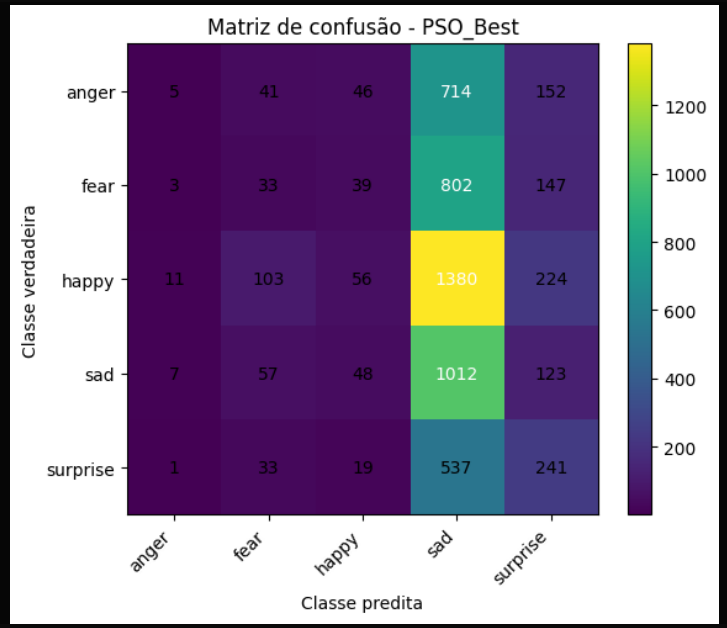
Resultados

Val Accuracy	Test Accuracy
0.8017	0.231

Top 3 Melhores Combinações

Evaluation	Iteration	Particle	Hidden Units	Dropout	Learning Rate
3	1	3	448	0.3261	0.0006983
1	1	1	384	0.3075	0.000355
8	2	3	448	0.32613	0.000698

Matriz de Confusão




```python
# fórmula standard do PSO
vel_new = (
    w * vel
    + c1 * r1 * (pbest - pos)
    + c2 * r2 * (gbest - pos)
)
pos_new = pos + vel_new

# garantir que as novas posições estão dentro dos limites
pos_new[0] = clip(pos_new[0], HU_MIN, HU_MAX)
pos_new[1] = clip(pos_new[1], D_MIN, D_MAX)
pos_new[2] = clip(pos_new[2], LOG_LR_MIN, LOG_LR_MAX)

particles_vel[p] = vel_new
particles_pos[p] = pos_new
```

Comparação Final Entre Modelos

Análise comparativa dos quatro modelos e impacto do domain shift nos resultados

Resultados Finais no Conjunto de Teste

Modelo	Test Accuracy	Test Loss
Default Transfer Learning	**24,07%**	2,8730
Grid Search Best	23,55%	3,3766
Random Search Best	23,52%	3,7732
PSO Best	23,09%	3,3352

Impacto do Domain Shift

- Validação (sintética): ~79%
- Teste (real): ~24%
- Perda significativa de desempenho

Principais Observações

- Modelo Default com Transfer Learning teve melhor desempenho
- Otimização não compensou a diferença entre domínios
- Balanceamento dos dados é mais crítico que ajuste de hiperparâmetros
- Todos os modelos continuaram a favorecer classes mais representadas

```python
# carregar modelo
model = load_model(model_path)
model.summary()
test_gen.reset()
test_loss, test_acc = model.evaluate(test_gen, verbose=1)

print(f"\nDesempenho no conjunto de teste - {nome_modelo}:")
print(f"  Test accuracy: {test_acc:.4f}")
print(f"  Test loss:     {test_loss:.4f}")

# previsões
test_gen.reset()
y_prob = model.predict(test_gen, verbose=1)
y_pred = np.argmax(y_prob, axis=1)

# matriz de confusão
cm = confusion_matrix(y_true, y_pred)

# relatório detalhado por classe
print("\nRelatório de classificação para o modelo:", nome_modelo)
print(classification_report(y_true, y_pred, target_names=class_names, digits=4))

# guardar gráfico da matriz de confusão
fig_filename = f"meta3_confusion_{nome_modelo}.png"
fig_path = os.path.join(FIG_DIR, fig_filename)
plot_confusion_matrix(
    cm,
    class_names,
    title=f"Matriz de confusão - {nome_modelo}",
    fig_path=fig_path
)

# guardar resultados numéricos numa lista
resultados.append({
    "modelo": nome_modelo,
    "descricao": descricao,
    "ficheiro_modelo": ficheiro_modelo,
    "test_accuracy": float(test_acc),
    "test_loss": float(test_loss)
})
```

Demonstração da Aplicação Web

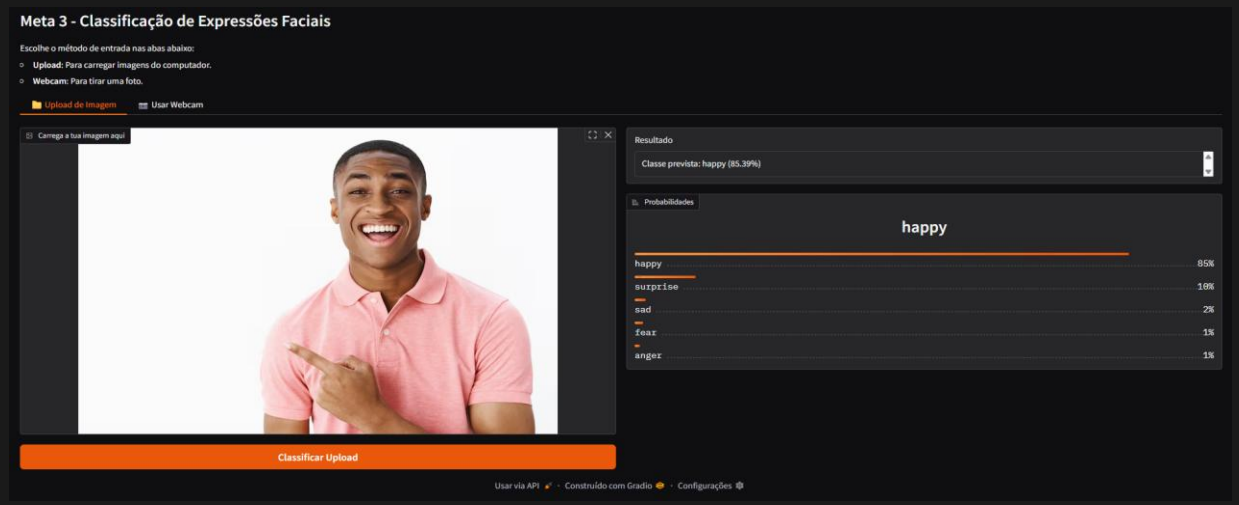
Funcionalidades

- Upload de imagens do computador
- Captura em tempo real via webcam
- Classificação das emoções com probabilidades
- Interface intuitiva e responsiva

</> Justificação

Modelo **Default com Transfer Learning** escolhido por ter apresentado o melhor desempenho, com accuracy de 24,07%.

Classificador de Emoções Faciais



</> Função de Predição

```python
def predict_image(image):

    if image is None:
        msg = (
            "Nenhuma imagem capturada. "
            "Clica no ícone da câmara para tirar a foto e só depois em Submit."
        )
        probs_zero = {cls: 0.0 for cls in CLASS_NAMES}
        return msg, probs_zero

    if isinstance(image, np.ndarray):
        image = Image.fromarray(image)

    img_batch = preprocess_image(image)
```

```python
    preds = model.predict(img_batch)
    preds = preds[0]

    best_idx = int(np.argmax(preds))
    best_class = CLASS_NAMES[best_idx]

    probs = {cls: float(preds[i]) for i, cls in enumerate(CLASS_NAMES)}

    label_text = f"Classe prevista: {best_class} ({preds[best_idx] * 100:.2f}%)"

    return label_text, probs
```

Conclusões

Transfer Learning

Estável e eficiente, melhorando a extração de características mesmo com menos dados.

Limitações do Dataset

Qualidade e representatividade do dataset são cruciais para o desempenho do classificador.

Otimização

Melhorias na validação, mas limitações no conjunto de teste devido ao domain shift.

Web App

Funcional e rápida, demonstrando a integração do modelo num contexto prático.

Conclusão Principal

O tratamento de dados (**balanceamento de classes**) revelou-se mais crítico para o desempenho do que a otimização de hiperparâmetros.

Bibliografia

Documentação e Recursos Técnicos

TensorFlow Documentation
Transfer learning and fine tuning

Keras API Reference
Image preprocessing and MobileNetV2

Scikit Learn Documentation
Classification metrics and hyperparameter search

SwarmPackagePy
Library for swarm algorithms

Ferramentas e Conjuntos de Dados

Basic Human Emotions Dataset
Sintetic facial expressions

FER2013 Dataset
Facial Expression Recognition 2013

Gradio
Interface for machine learning models

Ferramentas de Apoio

ChatGPT (OpenAI)
https://chatgpt.com/share/69310fc4-b38c-8001-8a64-247192cb29d2

Referências Teóricas

Fichas práticas e materiais teóricos
Unidade curricular de Inteligência Computacional, ISEC, 2025/2026